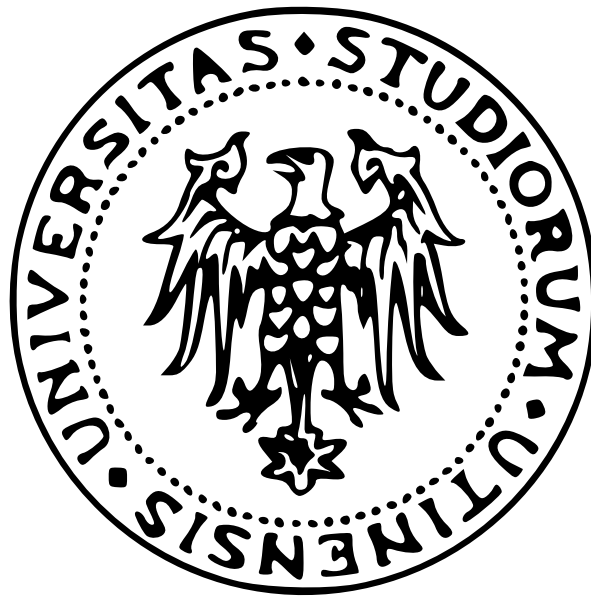




Relazione di progetto di Basi di Dati

167372	Nadal Mattia	nadal.mattia@spes.uniud.it
167947	Polesello Kevin	polesello.kevin@spes.uniud.it
168062	Rumiato Joshua	rumiato.joshua@spes.uniud.it
167024	Vignandel Francesco	vignandel.francesco@spes.uniud.it

Contents

Introduzione	3
1 Traccia del Progetto	3
1.1 Informazioni generali	3
1.2 Gestione partecipanti	3
1.3 Squadre e attività giornaliere	4
1.4 Gestione dei materiali	5
2 Glossario dei Termini	5
3 Analisi dei Requisiti	5
3.1 Gestione anagrafica dei partecipanti e degli alloggi	6
3.2 Pianificazione attività giornaliere, squadre e pasti	6
3.3 Organizzazione dei materiali	6
4 Assunzioni e Ipotesi Progettuali	6
5 Progettazione Concettuale (Schema ER)	7
6 Progettazione Logica	9
6.1 Analisi dei cicli	9
6.2 Analisi delle ridondanze	9
6.2.1 Stesura della tabella dei volumi	9
6.2.2 Operazioni interessate alla presenza di ridondanza	10
6.2.3 Soluzione 1: eliminazione della ridondanza	10
6.2.3.1 Operazione 1.	10
6.2.3.2 Operazione 2.	10
6.2.4 Soluzione 2: mantenimento dell'attributo derivato	11
6.2.4.1 Operazione 1.	11
6.2.4.2 Operazione 2.	11
6.2.5 Confronto finale e scelta progettuale	12
6.3 Eliminazione delle generalizzazioni	12
6.3.1 Prima generalizzazione: Attività	12
6.3.2 Seconda generalizzazione: Membro Equipe	13
6.3.3 Terza generalizzazione: Partecipante	13
6.4 Risultato della ristrutturazione dello schema	13
6.5 Scelte sui tipi di dato e sui domini degli attributi	15

7	Progettazione Fisica	17
7.1	Tipi enumerati e indici di sistema	17
7.2	Implementazione dei vincoli strutturali	17
7.3	Colonne Derivate e funzioni di generazione	18
7.4	Analisi delle scelte di indicizzazione e chiavi	19
7.4.1	Ottimizzazione tramite indici aggiuntivi	19
7.4.1.1	Indice funzionale case-insensitive sui partecipanti.	20
7.4.1.2	Indice funzionale temporale (GiST) sulle attività.	20
7.4.1.3	Indice d'integrità sulla chiave esterna cuochi-pasti.	21
7.5	Trigger e vincoli procedurali	21
7.5.1	Gestione dell'esclusività dei ruoli	21
7.5.2	Vincolo di sequenzialità temporale delle attività	23
7.5.3	Vincolo di presenza dell'animatore responsabile nelle stanze degli animati . .	24
7.5.4	Ulteriori vincoli aziendali e di consistenza	25
7.6	Implementazione delle query ed esempi d'uso	26
7.6.1	Inserimento di un nuovo animato.	26
7.6.2	Inserimento di una nuova attività di routine.	27
7.6.3	Coppie di routine che condividono un materiale.	27
7.6.4	Cuochi che hanno supervisionato un pasto con un allergene presente in una sola pietanza.	28
7.6.5	Animatore/i che ha supervisionato la routine con la maggiore quantità totale di materiale.	28
7.6.6	Animatori che hanno supervisionato solo attività senza materiali.	29
7.6.7	Aggiornamento dell'orario di una routine.	29
7.6.8	Cancellazione di una routine.	30
8	Conclusioni e sviluppi futuri	30

Introduzione

Il presente documento descrive le fasi di realizzazione del progetto di laboratorio del corso di Basi di Dati, partendo dall'analisi dei requisiti fino ad arrivare alla verifica del corretto funzionamento di quanto progettato.

In particolare, la relazione illustrerà i principali problemi affrontati e le soluzioni individuate nel corso della progettazione e dell'implementazione della base di dati richiesta.

1 Traccia del Progetto

1.1 Informazioni generali

L'obiettivo del progetto è la realizzazione di una base di dati in grado di gestire un **campo-scuola** negli aspetti che lo caratterizzano, supportando tutte le operazioni necessarie alla sua organizzazione.

Il campo è caratterizzato da un **edificio** (identificato tramite il proprio indirizzo) e da una lista di **partecipanti**. Per ogni partecipante vanno tracciati:

- Codice fiscale;
- Foto documento;
- Nome;
- Cognome;
- Sesso;
- Data di nascita;
- Luogo di nascita;
- Intolleranze alimentari;
- Note aggiuntive (qualsiasi informazione sanitaria, organizzativa o personale rilevante per la gestione del partecipante).

1.2 Gestione partecipanti

I partecipanti sono suddivisi in **equipe** e **animati**. L'*equipe* è composta da **cuochi**, **animatori**, **capi-campo** (animatori con speciali responsabilità) e **collaboratori**. Per ognuno di questi è rilevante conoscere:

- Numero di telefono ed e-mail personale;
- Valutazione sul suo operato, da inserire alla fine del campo, espressa come voto da 1 a 5 accompagnato da un commento.

Per quanto riguarda gli **animati**, invece, i contatti da salvare (numero di telefono ed e-mail) sono quelli dei genitori. Non sono previsti per loro meccanismi di valutazione.

Tutti i partecipanti (equipe + ragazzi) dormono nelle **camere** della struttura che ospita il campo. Le camere che ospitano animati *non possono essere miste*: i ragazzi dormono con altri ragazzi, e le ragazze altrettanto. Ogni camera ha:

- Un numero identificativo;
- Un numero di posti letto;
- Il piano dell'edificio in cui è collocata.

Le camere in cui dormono animati hanno inoltre un animatore responsabile (animatori maschi nelle camere maschili, animatrici nelle camere femminili). Il resto dell'*equipe* può dormire insieme.

Nessuna camera può contenere più persone del numero di posti letto indicati per essa.

1.3 Squadre e attività giornaliere

Gli animati vengono divisi in **squadre** per la partecipazione alle attività organizzate. Ogni squadra possiede:

- Un nome;
- Uno slogan;
- Un animatore responsabile;
- Punteggio cumulato delle attività di routine.

Le squadre si mettono inoltre a disposizione per lo svolgimento di alcune faccende domestiche.

Tutte le **attività** giornaliere sono caratterizzate da:

- Orario di inizio;
- Orario di fine;
- Eventuale descrizione;
- Categoria.

Le categorie sono organizzate come di seguito:

- **Pasti**
 - Colazione
 - Pranzo
 - Merenda
 - Cena
- **Faccende**
 - Preparazione tavoli colazione/pranzo/cena
 - Sgombero tavoli colazione/pranzo/cena
 - Pulizia stoviglie colazione/pranzo/cena
- **Routine**
 - Sveglia mattutina
 - Attività fisica
 - Attività della mattina
 - Attività del pomeriggio
 - Docce
 - Gioco libero
 - Attività serale
 - Resoconto della giornata
- **Extra**
 - Altro tipo di attività, non previste nelle categorie precedenti.

Tutte le squadre partecipano a tutte le attività di routine, che sono caratterizzate da un nome, un responsabile e una valutazione (solo testuale).

Ci deve essere una rigida sequenza tra tutti gli orari delle attività.

Ogni pasto viene preparato da tutti i cuochi (fra i quali c'è però un responsabile) e viene consumato da tutti i partecipanti al campo insieme. Ad ogni pasto possono essere servite più pietanze, definite da:

- Nome
- Descrizione
- Allergeni contenuti

1.4 Gestione dei materiali

Il campo prevede anche una lista di **materiali** necessari per il suo svolgimento. Ogni voce di questa lista deve registrare:

- Nome del materiale;
- Quantità;
- Unità di misura;
- Numero di utilizzi.

Fra i materiali necessari, alcuni potrebbero dover essere portati in autonomia dai partecipanti. Infine, esistono materiali specifici che vengono impiegati nello svolgimento di attività di routine particolari.

2 Glossario dei Termini

Table 1: Glossario dei termini di dominio

Termine	Definizione
Campo-scuola	L'evento organizzato oggetto del sistema, caratterizzato da una struttura ospitante, un insieme di partecipanti e un programma di attività.
Partecipante	Individuo che prende parte al campo. Può essere un animato o un membro dell'equipe.
Equipe	Insieme di animatori, cuochi e collaboratori che organizzano e gestiscono il campo.
Animatore	Membro dell'equipe che progetta e conduce attività ludiche, educative o ricreative per i ragazzi. Può essere responsabile di una camera o di una squadra.
Capo-campo	Animatore con responsabilità organizzative aggiuntive.
Cuoco	Membro dell'equipe responsabile della preparazione dei pasti. Può essere responsabile di uno o più pasti nel corso del campo.
Collaboratore	Membro dell'equipe con ruolo di supporto, non classificato come animatore né cuoco.
Animato	Ragazzo/ragazza partecipante al campo in qualità di destinatario delle attività.
Attività	Qualsiasi evento schedulato durante il campo, con orario di inizio, orario di fine e categoria.
Materiale	Risorsa fisica/oggetto necessaria/o allo svolgimento delle attività del campo, con nome, quantità, unità di misura e numero di utilizzi. Alcuni materiali possono dover essere portati in autonomia dagli animati.

3 Analisi dei Requisiti

La specifica dei requisiti descrive una realtà complessa e articolata, caratterizzata da numerose interdipendenze da considerare in fase di modellazione. Prima di poter sviluppare il Diagramma Entità-Relazione (ER) e procedere con le successive fasi della progettazione (logica e fisica), si è scelto di raggruppare le informazioni di interesse in tre macroaree funzionali.

3.1 Gestione anagrafica dei partecipanti e degli alloggi

Questa macroarea si occupa di censire la struttura ospitante e tutti i soggetti coinvolti nel campo scuola, definendone sia le generalità che aspetti più specifici (come quelli sanitari).

L'edificio ospitante il campo è caratterizzato solo dal proprio indirizzo ed è composto da stanze numerate. Non rappresenta la maggior fonte di complessità di questa sezione.

I partecipanti, invece, costituiscono uno dei principali nuclei informativi del sistema. Sono articolati in più categorie differenti, ognuna con precisi compiti e responsabilità. Va posta molta attenzione al pernottamento di queste figure al campo, in quanto vanno tenuti in considerazione specifici vincoli di capienza, responsabilità e sesso.

3.2 Pianificazione attività giornaliere, squadre e pasti

Questa sezione copre un altro fulcro della realtà di riferimento, ovvero le attività che scandiscono le giornate del campo. Pur presentando alcuni attributi comuni, queste si differenziano tra loro per natura e scopo.

Il sistema deve essere in grado di gestire il fatto che alcune di queste attività, specialmente fra quelle di routine e le faccende, vengano svolte in squadra. Inoltre, occorre garantire un vincolo di sequenzialità temporale rigida tra le attività del palinsesto, impedendo la sovrapposizione degli orari di inizio.

Non va infine trascurata la gestione delle responsabilità riguardanti i pasti e le attività di routine. Mentre le prime sono di competenza dei cuochi del campo, le seconde vanno assegnate a degli animatori. La responsabilità di una attività si può attribuire anche al/ai capo/i-campo.

3.3 Organizzazione dei materiali

Predisporre la gestione dell'inventario di beni necessari al corretto svolgimento del campo, monitorandone la disponibilità e la destinazione d'uso.

Il sistema deve poter distinguere tra i materiali forniti dall'organizzazione e quelli che devono essere portati da casa dai partecipanti (in autonomia). Va considerata anche la gestione delle quantità, dei possibili riutilizzi e del legame diretto tra specifici materiali e le attività di routine che ne richiedono obbligatoriamente l'impiego.

4 Assunzioni e Ipotesi Progettuali

Nonostante il discreto livello di dettaglio offerto dalla traccia proposta, si rende necessaria una fase preliminare di analisi del testo volta a individuare e a risolvere alcune fonti di ambiguità o lacune informative.

Di seguito vengono elencate le principali ipotesi progettuali e le assunzioni integrative formulate:

- Si assume che il campeggio sia svolto interamente in Italia, in modo da facilitare e uniformare la memorizzazione dell'indirizzo dell'edificio (o degli edifici) ospitanti il campo;
- Il ruolo dei partecipanti (animatori, animati, cuochi, ecc.) è univoco e rimane invariato per la durata del campo;
- Nel caso in cui più animatori dormono in una stanza con degli animati ognuno è da considerare responsabile della camera, così da non dover tracciare nel dettaglio responsabilità non di primaria importanza;

- Si assume che la composizione delle squadre sia statica per tutta la durata del campo scuola e che ogni animato appartenga a una e una sola squadra;
- I materiali da portare in autonomia si riferiscono ad oggetti richiesti a tutti gli animati, e non a singole persone;
- I punteggi assegnati alle squadre sono soggetti unicamente a incrementi; non sono previsti decrementi.

5 Progettazione Concettuale (Schema ER)

Seguitamente all'analisi dei requisiti svolta, si è costruito lo schema ER adottando un approccio top-down a partire dalle macroaree funzionali definite. Di seguito viene mostrato lo schema globale. Si ritiene doveroso ricordare che questo è un diagramma concettuale, che astrae dall'implementazione fisica e dalle tecnologie utilizzate. Tutti i vincoli non esprimibili graficamente, dovuti a limitazioni del modello, verranno elencati nel capitolo dedicato ai vincoli (trigger).

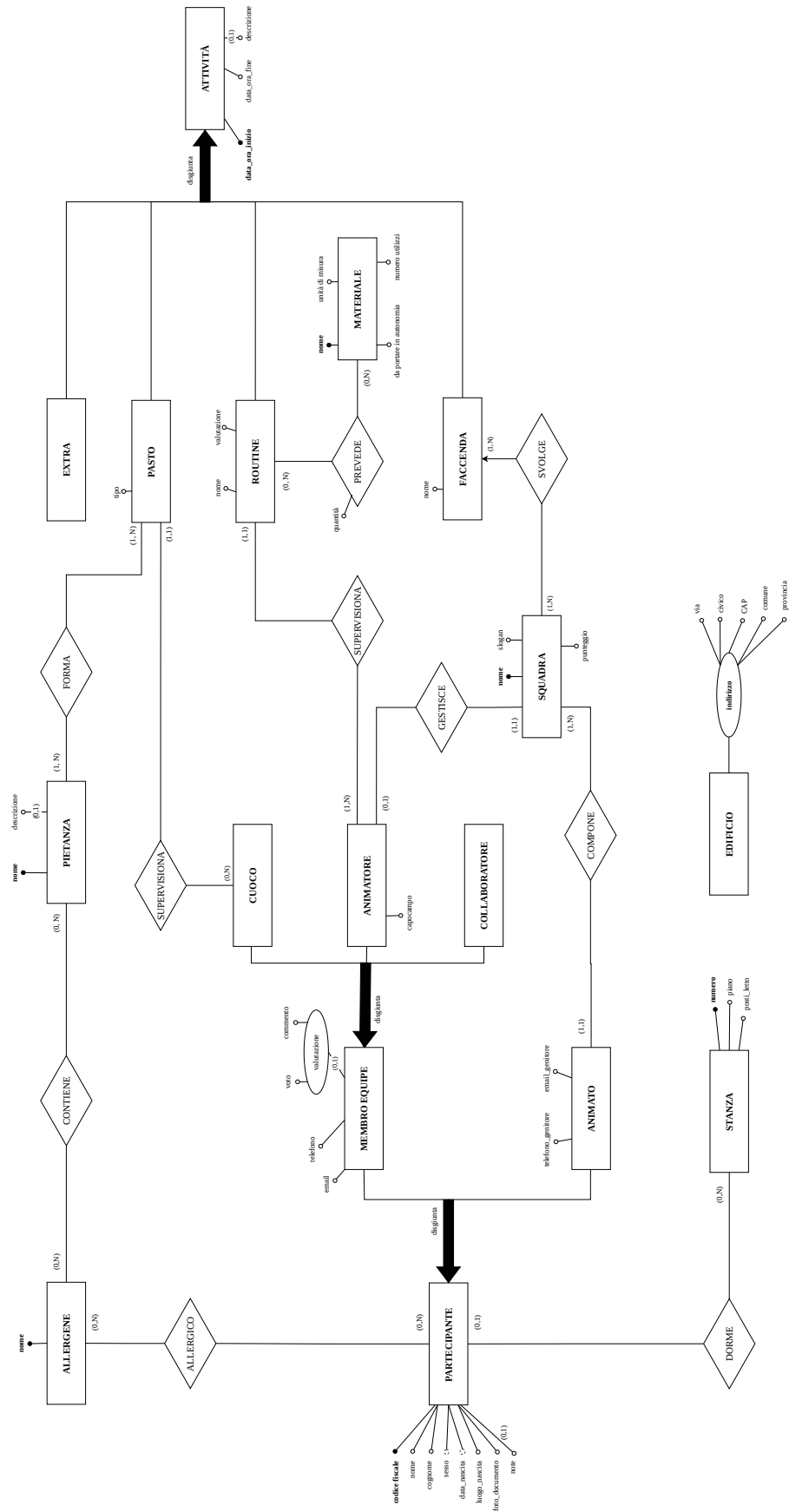


Figure 1: Schema Entità-Relazione

6 Progettazione Logica

Questa fase ha l'obiettivo di definire il modello logico dei dati, tenendo conto della tipologia di DBMS su cui verrà implementato il database (nel caso in esame, un RDBMS).

Per adattare lo schema ER al modello relazionale, è necessario effettuare alcune operazioni di analisi e modifica nello schema originale.

6.1 Analisi dei cicli

L'analisi del modello precedentemente illustrato non evidenzia la presenza di cicli strutturali tra entità in senso stretto. Sono tuttavia presenti alcune configurazioni degne di nota che coinvolgono relazioni di generalizzazione/specializzazione:

- **Responsabilità relative ai pasti:** coinvolge le relazioni Cuoco – Membro equipe – Partecipante – Allergene – Pietanza – Pasto. È possibile definire un vincolo che impedisca ad un cuoco di supervisionare un pasto che contiene pietanze a cui è allergico.
- **Supervisione delle attività:** un animatore può gestire attività di routine a cui partecipa la squadra che gestisce. Ciò non viene considerato un problema dal momento che la traccia non evidenzia come critiche situazioni di questo genere.

6.2 Analisi delle ridondanze

In questa fase si valuta se mantenere o eliminare la ridondanza che verrebbe introdotta considerando un attributo `quantita_totale` nell'entità `Materiale`, che può essere derivato a partire dalle attività di routine sommandone le quantità associate di ciascun materiale.

Per quantificare i costi in modo realistico, si assume la convenzione secondo cui un accesso in scrittura abbia un costo doppio rispetto a un accesso in lettura alla base di dati.

6.2.1 Stesura della tabella dei volumi

La tabella dei volumi costituisce uno strumento della progettazione logica che consente di stimare le dimensioni della base di dati in condizioni operative ordinarie, esprimendo il numero atteso di record per ciascuna entità o relazione. Le stime ottenute rappresentano la base per l'analisi della ridondanza considerata.

Per brevità, la tabella viene costruita limitatamente alle entità e alle relazioni strettamente coinvolte nell'analisi da condurre:

- **Materiale:** si stima che, per un campo scuola, siano necessari circa 50 materiali distinti.
- **Routine:** dalla traccia si individuano otto attività di routine giornaliere. Assumendo una durata media del campo pari ad una settimana, si ottengono complessivamente $7 \cdot 8 = 56$ istanze di attività di routine.
- **Prevede:** si ipotizza che, mediamente, quattro attività di routine al giorno richiedano l'utilizzo di tre materiali differenti, mentre le restanti non prevedano alcun materiale. Considerando un campo-scuola di una settimana, il numero complessivo di istanze della relazione risulta pari a $4 \cdot 3 \cdot 7 = 84$.

Table 2: Tabella dei volumi

Concetto	Tipo	Volume
Materiale	Entità	50
Prevede	Relazione	84
Routine	Entità	56

6.2.2 Operazioni interessate alla presenza di ridondanza

Per valutare la convenienza del mantenimento della ridondanza vengono considerate le principali operazioni che interesserebbero gli attributi *quantita* (in *Prevede*) e *quantita_totale* (in *Materiale*). Per ciascuna operazione viene stimata la frequenza di esecuzione e il relativo costo in termini di accessi alla base di dati.

Table 3: Operazioni interessate

ID	Operazione	Frequenza
1	Inserimento di una nuova attività di routine	56 volte
2	Visualizzare tutto il materiale necessario per il campo	1 volta (svolta come controllo prima dell'inizio del campo)

6.2.3 Soluzione 1: eliminazione della ridondanza

La prima soluzione consiste nell'eliminare la ridondanza, mantenendo l'informazione relativa alla quantità esclusivamente nella relazione *Prevede*.

6.2.3.1 Operazione 1.

In questo scenario, durante l'inserimento di una nuova attività di routine, è necessario inserire dapprima la nuova istanza dell'entità *Routine* e successivamente creare, in media, tre nuove istanze della relazione *Prevede*, assumendo che i materiali siano già presenti nel sistema.

Table 4: Soluzione 1 – Accessi per l'Operazione 1

Concetto	Tipo	Accessi	Costo	Tipo accesso
Routine	Entità	1	2	Scrittura
Prevede	Relazione	3	2	Scrittura

Poiché questa operazione viene eseguita 56 volte nel corso della preparazione di un campo, il costo complessivo risulta pari a:

$$56 \cdot (3 + 1) \cdot 2 = 448,$$

dove 3 rappresenta il numero medio di inserimenti nella relazione *Prevede*, 1 l'inserimento nell'entità *Routine*, mentre 2 rappresenta il costo attribuito a ciascun accesso in scrittura.

6.2.3.2 Operazione 2.

Per ottenere la quantità complessiva di materiale necessaria al campo è necessario leggere tutte le istanze della relazione *Prevede* e aggregare i valori corrispondenti.

Table 5: Soluzione 1 – Accessi per l'Operazione 2

Concetto	Tipo	Accessi	Costo	Tipo accesso
Materiale	Entità	1	1	Lettura
Prevede	Relazione	84/56=1.5	1	Lettura

Dalla tabella si ricava intuitivamente il costo dell'operazione:

$$1 \cdot (1 + 1.5) \cdot 1 = 2.5.$$

6.2.4 Soluzione 2: mantenimento dell'attributo derivato

La seconda soluzione prevede il mantenimento dell'attributo derivato `quantita_totale` all'interno dell'entità `Materiale`.

6.2.4.1 Operazione 1.

In questo caso, oltre all'inserimento della nuova attività di routine e delle relative istanze della relazione `Prevede`, è necessario aggiornare anche il valore dell'attributo `quantita_totale` di ciascun materiale coinvolto. Questo si traduce in un numero maggiore di accessi in lettura e scrittura alle tabelle per singolo inserimento.

Table 6: Soluzione 2 – Accessi per l'Operazione 1

Concetto	Tipo	Accessi	Costo	Tipo accesso
Routine	Entità	1	2	Scrittura
Prevede	Relazione	3	2	Scrittura
Materiale	Entità	3	1	Lettura
Materiale	Entità	3	2	Scrittura

Il costo complessivo per 56 attività (ovvero una settimana di campo) è pari a:

$$56 \cdot [(3 + 3 + 1) \cdot 2 + 3 \cdot 1] = 952.$$

6.2.4.2 Operazione 2.

La presenza dell'attributo derivato semplifica la consultazione del materiale necessario per il campo. Infatti, non è più necessario ricavare il valore mediante l'aggregazione delle istanze di `Prevede`, poiché esso è già memorizzato in `Materiale`. L'operazione richiede quindi un solo accesso in lettura all'entità `Materiale`.

Table 7: Soluzione 2 – Accessi per l'Operazione 2

Concetto	Tipo	Accessi	Costo	Tipo accesso
Materiale	Entità	1	1	Lettura

Il calcolo del costo diventa immediato: $1 \cdot 1 \cdot 1 = 1$.

6.2.5 Confronto finale e scelta progettuale

Dal confronto tra le due soluzioni emerge che l'eliminazione della ridondanza comporta un costo complessivo pari a $448 + 2.5 = 450.5$, mentre il mantenimento dell'attributo derivato determina un costo totale più che duplicato: $952 + 1 = 953$.

Sebbene la seconda soluzione consenta di ridurre il costo delle operazioni di consultazione, essa incrementa in maniera significativa il costo delle operazioni di aggiornamento, che risultano molto più frequenti nello scenario considerato.

Alla luce delle stime effettuate, si ritiene quindi preferibile **eliminare la ridondanza** e calcolare il valore dell'attributo quantità solo quando necessario, evitando di memorizzare un dato derivabile e mantenendo così una struttura più semplice e coerente.

6.3 Eliminazione delle generalizzazioni

Il modello relazionale non consente di rappresentare direttamente le generalizzazioni e le specializzazioni presenti nello schema Entità-Relazione. È pertanto necessario trasformarle adottando una delle strategie di ristrutturazione previste. Poiché tutte le generalizzazioni individuate nello schema sono totali e disgiunte, è possibile ricorrere a tre diverse soluzioni.

1. **Accorpamento delle entità figlie nel genitore:** le entità figlie vengono eliminate e i loro attributi vengono incorporati nell'entità padre. L'appartenenza a una particolare specializzazione può essere rappresentata mediante un attributo discriminante oppure osservando quali attributi risultano valorizzati. Questa soluzione è indicata quando la maggior parte delle operazioni coinvolge l'entità padre, il numero di specializzazioni è contenuto e ciascuna di esse introduce pochi attributi aggiuntivi.
2. **Accorpamento del genitore nelle entità figlie:** l'entità padre viene eliminata e i suoi attributi vengono replicati in ciascuna entità figlia. Questa strategia risulta conveniente quando gli accessi riguardano prevalentemente le entità figlie, il padre possiede un numero limitato di attributi e la generalizzazione è totale, in modo da evitare la presenza di dati inutilmente duplicati.
3. **Sostituzione della generalizzazione mediante relazioni:** l'entità padre viene mantenuta e ciascuna entità figlia viene collegata ad essa tramite una relazione identificante. In questo modo le entità figlie risultano identificate dalla chiave primaria del padre. Tale soluzione è particolarmente adatta quando gli accessi al padre e alle specializzazioni sono distinti e si desidera evitare la presenza di numerosi valori NULL, tipica della prima strategia.

Qualora le modalità di accesso alle diverse specializzazioni siano differenti, è inoltre possibile adottare soluzioni ibride, applicando strategie diverse a generalizzazioni differenti.

Nell'ambito dello schema ER relativo alla traccia proposta sono presenti tre generalizzazioni.

6.3.1 Prima generalizzazione: Attività

La prima riguarda l'entità Attività, specializzata nelle entità Routine, Pasto, Faccenda ed Extra. È stata adottata la strategia di **accorpamento del genitore nelle entità figlie**, per le seguenti motivazioni:

- l'entità Attività non partecipa direttamente ad altre relazioni;
- le relazioni coinvolgono separatamente le singole specializzazioni;
- la generalizzazione è totale e disgiunta;
- il padre possiede solamente gli attributi comuni alle attività (orario di inizio, orario di fine ed eventuale descrizione).

Di conseguenza, gli attributi dell'entità Attività vengono replicati nelle entità figlie, eliminando la

necessità di mantenere il padre.

6.3.2 Seconda generalizzazione: Membro Equipe

La seconda generalizzazione riguarda l'entità Membro equipe, specializzata nelle entità Animatore, Cuoco e Collaboratore. Anche in questo caso è stata scelta la strategia di **accorpamento del genitore nelle entità figlie**, poiché:

- Membro equipe non è direttamente coinvolta in altre relazioni;
- gli accessi avvengono principalmente attraverso le singole specializzazioni;
- la generalizzazione è totale e disgiunta;
- mantenere l'entità padre comporterebbe la presenza di un'entità priva di attributi propri realmente significativi, introducendo un livello di complessità non necessario.

6.3.3 Terza generalizzazione: Partecipante

L'ultima generalizzazione riguarda l'entità Partecipante, specializzata nelle entità Animato e Membro equipe. In questo caso è stata adottata la strategia di **sostituzione della generalizzazione mediante relazioni**. La scelta è motivata dal fatto che le due specializzazioni presentano numerosi attributi differenti e sono interessate da operazioni di accesso diverse. Mantenere un'unica entità avrebbe comportato la presenza di molti attributi non valorizzati.

Le entità Animato e Membro equipe vengono pertanto identificate dalla chiave primaria dell'entità Partecipante, dalla quale ereditano l'identificatore.

6.4 Risultato della ristrutturazione dello schema

Le analisi e gli interventi descritti nelle sezioni precedenti hanno portato alla realizzazione di una versione aggiornata dello schema Entità-Relazione, che costituisce la base per le successive fasi della progettazione ed è illustrata nella figura seguente. Sono inoltre state trasformate in tabelle le relazioni molti a molti e gli attributi composti sono stati decomposti nei rispettivi attributi atomici.

6.5 Scelte sui tipi di dato e sui domini degli attributi

In fase di traduzione dello schema logico in schema fisico, per ciascun attributo è stato selezionato il tipo di dato PostgreSQL più adeguato a rappresentarne correttamente il dominio, bilanciando esigenze di correttezza semantica, efficienza di memorizzazione e facilità di validazione.

Per quanto riguarda gli attributi di tipo testuale legati a documenti o immagini, come `foto_documento` dell'entità Partecipante, si è scelto di adottare il tipo `VARCHAR`, poiché il campo non memorizza direttamente il file binario, bensì il percorso (*path*) tramite cui recuperare la foto dal file system o da uno storage esterno. Questa scelta evita di appesantire la base di dati con contenuti binari di grandi dimensioni, delegando la gestione fisica dei file a un livello applicativo separato, più adatto a questo tipo di risorse.

Per gli attributi temporali che identificano l'inizio e la fine delle attività (`data_ora_inizio`, `data_ora_fine`), è stato adottato il tipo `TIMESTAMP` senza fuso orario. Questa decisione è motivata dal fatto che il campo si svolge interamente in un'unica località fisica e in un unico fuso orario (cfr. le assunzioni progettuali), rendendo superflua la gestione di conversioni tra fusi diversi; l'adozione di `TIMESTAMP` semplifica inoltre la logica dei trigger di sequenzialità temporale e degli indici basati su `TSRANGE`, che operano più naturalmente su valori privi di offset e che verranno spiegati successivamente.

Per gli attributi che rappresentano insiemi finiti e statici di valori ammessi, come la categoria delle attività (pasti, faccende, routine, extra), si è scelto di ricorrere a tipi enumerati (`enum`) personalizzati, anziché a semplici campi `VARCHAR` vincolati da `CHECK`. Questa soluzione non è tuttavia priva di controindicazioni: il dominio dei valori ammessi risulta poco flessibile, poiché l'aggiunta di un nuovo valore richiede un'istruzione DDL dedicata, e l'utilizzo degli `enum` all'interno di espressioni comparative complesse presenta alcune limitazioni.

7 Progettazione Fisica

La progettazione fisica ha l'obiettivo di tradurre lo schema logico relazionale in una struttura concretamente implementabile all'interno del RDBMS PostgreSQL. Questa fase definisce rigorosamente i tipi di dato, le estensioni dei domini, i vincoli di integrità fisici e i meccanismi procedurali necessari a garantire il rispetto delle regole applicative precedentemente individuate.

7.1 Tipi enumerati e indici di sistema

Per migliorare la qualità dei dati memorizzati sono stati introdotti tipi enumerati impiegati per rappresentare insiemi finiti e statici di valori ammessi, impedendo l'inserimento di stringhe invalide e ottimizzando lo spazio di memorizzazione su disco.

PostgreSQL gestisce internamente tali strutture mediante identificatori numerici sequenziali. In particolare, sono stati definiti i tipi `tipologia_pasto`, `tipologia_faccenda`, `tipologia_routine`.

7.2 Implementazione dei vincoli strutturali

I vincoli strutturali derivati dalla progettazione logica sono stati mappati sfruttando le clausole native di chiave primaria, chiave esterna, vincoli di unicità e vincoli CHECK locali.

Le chiavi esterne garantiscono l'integrità referenziale del database e, grazie alle regole di aggiornamento e cancellazione in cascata, impediscono la creazione di record orfani.

Ulteriori controlli di consistenza logica sono stati distribuiti tramite espressioni CHECK sulle singole tabelle. Tra i principali si segnalano:

- il controllo preventivo della struttura del codice fiscale del partecipante, validato tramite una funzione dedicata (`is_valid_cf`) che verifica, per mezzo di un'espressione regolare, la conformità del valore inserito al formato standard a 16 caratteri; da tale codice si derivano successivamente, tramite apposita funzione, data di nascita e sesso del partecipante;
- il controllo del formato degli indirizzi e-mail, effettuato tramite la funzione `is_valid_email`, anch'essa basata su un'espressione regolare che verifica la presenza di una struttura del tipo *utente@dominio.estensione*;
- il controllo del formato dei numeri di telefono, effettuato tramite la funzione `is_valid_phone_number`, che verifica tramite espressione regolare che il valore inserito sia composto esclusivamente da dieci cifre numeriche;
- il controllo che impedisce l'inserimento di valori negativi per il numero di posti letto di una stanza, il numero di utilizzi dei materiali e il numero di materiali previsti nell'entità Prevede.

L'utilizzo di funzioni IMMUTABLE scritte in linguaggio PL/pgSQL, richiamate all'interno dei vincoli CHECK, consente di centralizzare la logica di validazione basata su espressioni regolari, garantendo al contempo maggiore leggibilità e riutilizzabilità del codice rispetto all'inserimento diretto del pattern regex in ogni singolo vincolo.

```
CREATE FUNCTION is_valid_cf(text)
RETURNS boolean AS $$
BEGIN
    RETURN $1 ~ '^[A-Z]{6}[0-9]{2}[A-EHLMRST]{1}[0-9]{2}[A-Z]{1}[0-9]{3}[A-Z]{1}$';
END;
$$ LANGUAGE plpgsql IMMUTABLE;

CREATE FUNCTION is_valid_email(text)
RETURNS boolean AS $$
```

```

BEGIN
    RETURN $1 ~ '^[A-Za-z0-9._%+~@][A-Za-z0-9.-]+\.[A-Za-z]{2,}$';
END;
$$ LANGUAGE plpgsql IMMUTABLE;

CREATE FUNCTION is_valid_phone_number(text)
RETURNS boolean AS $$
BEGIN
    RETURN $1 ~ '^[0-9]{10}$';
END;
$$ LANGUAGE plpgsql IMMUTABLE;

CREATE FUNCTION is_valid_voto(int)
RETURNS boolean AS $$
BEGIN
    RETURN $1 IS NULL OR $1 BETWEEN 1 AND 5;
END;
$$ LANGUAGE plpgsql IMMUTABLE;

```

7.3 Colonne Derivate e funzioni di generazione

Per ottimizzare la gestione dei dati derivati dal codice fiscale, come il sesso e la data di nascita, il progetto utilizza colonne generate definite tramite la clausola `GENERATED ALWAYS AS` con memorizzazione dei valori. Questo approccio garantisce che i dati derivati rimangano sempre coerenti con la fonte primaria, ovvero il campo `codice_fiscale`, eliminando la necessità di aggiornamenti manuali.

Il vincolo `CHECK` basato su espressione regolare, descritto in precedenza, garantisce che il valore di `codice_fiscale` sia sempre strutturalmente valido: le funzioni di derivazione operano quindi su un input la cui conformità è già verificata in fase di inserimento.

Le colonne generate implementate sono:

- **sesso**: ottenuto tramite la funzione `cf_to_sesso(codice_fiscale)`, che determina il genere a partire dal codice fiscale secondo le regole fiscali italiane.
- **data_nascita**: calcolata tramite la funzione `cf_to_birthdate(codice_fiscale)`, che estrae e ricostruisce la data di nascita dal codice fiscale.

Poiché il codice fiscale, una volta inserito, difficilmente viene modificato, le funzioni di derivazione (`cf_to_sesso` e `cf_to_birthdate`) vengono eseguite una sola volta in fase di scrittura del dato. Questa scelta progettuale privilegia l'uso di spazio su disco aggiuntivo rispetto al ricalcolo delle informazioni ad ogni interrogazione, con l'obiettivo di ridurre il carico computazionale durante le interrogazioni.

```

CREATE OR REPLACE FUNCTION cf_to_birthdate(cf CHAR(16))
RETURNS DATE
LANGUAGE sql
IMMUTABLE
AS $$
    SELECT to_date(
        (
            CASE
                WHEN substring(cf, 7, 2)::int <= 25 THEN '20'
                ELSE '19'
            END
            || substring(cf, 7, 2)
        )
    );

```

```

)
||
CASE substring(cf, 9, 1)
  WHEN 'A' THEN '01'
  WHEN 'B' THEN '02'
  WHEN 'C' THEN '03'
  WHEN 'D' THEN '04'
  WHEN 'E' THEN '05'
  WHEN 'H' THEN '06'
  WHEN 'L' THEN '07'
  WHEN 'M' THEN '08'
  WHEN 'P' THEN '09'
  WHEN 'R' THEN '10'
  WHEN 'S' THEN '11'
  WHEN 'T' THEN '12'
END
||
CASE
  WHEN substring(cf, 10, 2)::int > 40
    THEN lpad((substring(cf, 10, 2)::int - 40)::text, 2, '0')
    ELSE substring(cf, 10, 2)
END,
'YYMMDD'
);
$$;

```

```

CREATE OR REPLACE FUNCTION cf_to_sesso(cf CHAR(16))
RETURNS CHAR(1)
LANGUAGE sql
IMMUTABLE
AS $$
  SELECT CASE
    WHEN substring(cf, 10, 2)::int > 40 THEN 'F'
    ELSE 'M'
  END;
$$;

```

7.4 Analisi delle scelte di indicizzazione e chiavi

L'analisi delle scelte di indicizzazione e delle chiavi evidenzia come l'ottimizzazione mediante indici rivesta un ruolo fondamentale nel garantire prestazioni elevate e un'efficiente esecuzione delle query del database relazionale progettato. In particolare, la definizione di indici strategici su attributi frequentemente coinvolti in operazioni di ricerca, join o filtro consente di ridurre significativamente i tempi di esecuzione delle query, migliorando l'usabilità del sistema.

7.4.1 Ottimizzazione tramite indici aggiuntivi

PostgreSQL indicizza automaticamente tutte le chiavi primarie delle tabelle e, poiché i volumi previsti sono contenuti (inferiori a 200 righe), non è stato ritenuto necessario introdurre ulteriori indici, che avrebbero potuto incrementare il costo delle operazioni di aggiornamento.

Per alcune operazioni ritenute particolarmente rilevanti o ricorrenti si è comunque scelto di introdurre tre indici aggiuntivi, descritti di seguito.

7.4.1.1 Indice funzionale case-insensitive sui partecipanti.

Struttura utilizzata: B-Tree.

All'interno dell'applicazione, l'utente potrà effettuare ricerche senza distinguere tra lettere maiuscole e minuscole (es. cercando "marco" o "GIRALDI"), garantendo ricerche esatte in tempo logaritmico $O(\log N)$.

```
-- Creazione dell'indice funzionale composto
CREATE INDEX idx_partecipanti_nome_cognome_lower
ON partecipanti ((LOWER(nome)), (LOWER(cognome)));

-- Query di esempio che sfrutta appieno l'indice:
SELECT * FROM partecipanti
WHERE LOWER(nome) = 'mArCo' AND LOWER(cognome) = 'gIrAlDi';
```

7.4.1.2 Indice funzionale temporale (GiST) sulle attività.

Struttura utilizzata: GiST (Generalized Search Tree).

Gli indici classici, come quelli basati su B-Tree, organizzano i dati in modo ordinato e lineare lungo una dimensione. Questo li rende molto efficienti per confronti semplici, come verificare se un valore è maggiore, minore o uguale a un altro, oppure per ricerche su intervalli unidimensionali. Tuttavia, diventano meno adatti quando si devono gestire condizioni più complesse, come la sovrapposizione tra intervalli di tempo.

Per risolvere questo limite, PostgreSQL introduce tipi di dato più avanzati come TSRANGE, che rappresenta un intervallo temporale come un unico oggetto composto da un valore iniziale e uno finale. Questo consente di trattare i periodi di tempo non come singoli valori separati, ma come entità complete su cui è possibile eseguire operazioni specifiche.

Grazie a TSRANGE è possibile effettuare in modo diretto e ottimizzato operazioni come la verifica di sovrapposizione tra intervalli, rendendo le query più espressive e spesso anche più efficienti rispetto all'uso della funzione OVERLAPS. In questo modo il database può sfruttare strutture dati e operatori dedicati, migliorando sia la leggibilità del codice SQL sia le prestazioni nelle operazioni su dati temporali.

```
-- 1. Abilita l'estensione nativa di PostgreSQL per usare i tipi btree dentro l'
    -- albero GIST
CREATE EXTENSION IF NOT EXISTS btree_gist;

-- 2. Creazione dell'indice basato sul range temporale per le tabelle delle attività
CREATE INDEX idx_attivita_routine_intervallo
ON attivita_routine USING GIST (tsrange(data_ora_inizio, data_ora_fine));

CREATE INDEX idx_attivita_faccende_intervallo
ON attivita_faccende USING GIST (tsrange(data_ora_inizio, data_ora_fine));

CREATE INDEX idx_pasti_intervallo
ON attivita_pasto USING GIST (tsrange(data_ora_inizio, data_ora_fine));

CREATE INDEX idx_attivita_extra_intervallo
ON attivita_extra USING GIST (tsrange(data_ora_inizio, data_ora_fine));
```

7.4.1.3 Indice d'integrità sulla chiave esterna cuochi-pasti.

Struttura utilizzata: B-Tree.

Utile al vincolo: “Un cuoco non può supervisionare un pasto se è allergico a un ingrediente contenuto in una pietanza servita in quel pasto”.

Quando viene inserito o modificato un pasto, il trigger deve effettuare una catena di JOIN a partire dal cuoco assegnato (`codice_fiscale_cuoco`) fino agli ingredienti e alle relative allergie. PostgreSQL non crea automaticamente gli indici sulle chiavi esterne (FOREIGN KEY): senza questo indice, per ogni controllo sulle allergie del cuoco associato a un determinato pasto, il database effettuerebbe un Sequential Scan sulla tabella pasti. Questo indice permette al database di isolare e controllare immediatamente solo i pasti di competenza di quel cuoco, salvaguardando le performance dei controlli d'integrità.

```
CREATE INDEX idx_pasti_cuoco_supervisore
ON attivita_pasto (codice_fiscale_cuoco_supervisore);
```

7.5 Trigger e vincoli procedurali

Alcuni vincoli di integrità non possono essere espressi mediante i tradizionali vincoli dichiarativi, poiché richiedono verifiche che coinvolgono più tabelle. Per questo motivo sono stati implementati trigger dedicati associati a funzioni scritte in linguaggio PL/pgSQL. In seguito, vengono illustrati quelli più importanti ed elencati i rimanenti.

7.5.1 Gestione dell'esclusività dei ruoli

La generalizzazione relativa alle tipologie di partecipante, risolta tramite la terza strategia di ristrutturazione (sostituzione mediante relazioni identificanti), non garantisce automaticamente nel modello relazionale che un partecipante compaia in una e una sola tra le tabelle animatori, cuochi, collaboratori e animati. Tale vincolo di totalità ed esclusività è stato quindi implementato proceduralmente tramite trigger.

La funzione `check_single_ruolo_insert_update` verifica, mediante una UNION ALL sulle quattro tabelle figlie, quante volte compare il codice fiscale del partecipante coinvolto nell'operazione. Se il conteggio risulta diverso da uno, l'operazione viene rifiutata: questo consente di rilevare sia il caso di un partecipante privo di ruolo, sia quello di un partecipante associato a più ruoli contemporaneamente. La funzione viene agganciata, tramite trigger omonimi, sia alla tabella `partecipanti` sia a ciascuna delle quattro tabelle figlie, poiché la violazione può originarsi indipendentemente da quale tabella viene modificata.

Poiché l'inserimento di un nuovo partecipante richiede due distinte istruzioni INSERT all'interno della stessa transazione (una su `partecipanti` e una sulla tabella figlia corrispondente), il controllo non può essere eseguito immediatamente dopo la prima istruzione, pena il rifiuto di operazioni valide. Per questo motivo i trigger sono stati definiti come CONSTRAINT TRIGGER DEFERRABLE INITIALLY DEFERRED: la verifica viene così posticipata al momento del commit della transazione, quando lo stato della base di dati è definitivo, garantendo che operazioni intermedie temporaneamente incoerenti non causino il rifiuto di transazioni complessivamente corrette.

```
CREATE OR REPLACE FUNCTION check_single_ruolo_insert_update()
RETURNS TRIGGER AS $$
DECLARE
    cnt INTEGER;
```

```

BEGIN
    SELECT COUNT(*) INTO cnt
    FROM (
        SELECT codice_fiscale FROM animatori
        UNION ALL
        SELECT codice_fiscale FROM collaboratori
        UNION ALL
        SELECT codice_fiscale FROM animati
        UNION ALL
        SELECT codice_fiscale FROM cuochi
    ) t
    WHERE codice_fiscale = NEW.codice_fiscale;

    IF cnt <> 1 THEN
        RAISE EXCEPTION 'Un partecipante deve avere sempre un unico ruolo';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE CONSTRAINT TRIGGER trg_partecipanti_ruolo_insert_update
AFTER INSERT OR UPDATE ON partecipanti
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW
EXECUTE FUNCTION check_single_ruolo_insert_update();

```

Analoga logica, ma simmetrica, è stata adottata per la cancellazione tramite la funzione `check_single_ruolo_delete`: dopo l'eliminazione di una riga, se il partecipante non risulta più presente in nessuna tabella figlia ma è ancora presente in `partecipanti`, l'operazione viene rifiutata, impedendo la creazione di partecipanti privi di specializzazione. Anche in questo caso i trigger sono differiti a fine transazione, per consentire la cancellazione coordinata delle righe su `partecipanti` e sulla relativa tabella figlia.

```

CREATE OR REPLACE FUNCTION check_single_ruolo_delete()
RETURNS TRIGGER AS $$
DECLARE
    cnt INTEGER;
BEGIN
    SELECT COUNT(*)
    INTO cnt
    FROM (
        SELECT codice_fiscale FROM animatori
        UNION ALL
        SELECT codice_fiscale FROM collaboratori
        UNION ALL
        SELECT codice_fiscale FROM animati
        UNION ALL
        SELECT codice_fiscale FROM cuochi
    ) t
    WHERE codice_fiscale = OLD.codice_fiscale;

    IF cnt > 1 THEN
        RAISE EXCEPTION 'Un partecipante deve avere sempre un unico ruolo';
    END IF;

```

```

IF cnt = 0 THEN
    SELECT COUNT(*)
    INTO cnt
    FROM partecipanti
    WHERE codice_fiscale = OLD.codice_fiscale;

    IF cnt <> 0 THEN
        RAISE EXCEPTION 'Un partecipante non puo rimanere senza ruolo';
    END IF;
END IF;

RETURN OLD;
END;
$$ LANGUAGE plpgsql;

```

7.5.2 Vincolo di sequenzialità temporale delle attività

La traccia richiede che le attività del campo si susseguano secondo una rigida sequenza temporale, senza che gli intervalli di due attività distinte possano sovrapporsi, indipendentemente dalla categoria di appartenenza (routine, faccende, pasti, extra). Trattandosi di un vincolo trasversale a più tabelle, non è esprimibile mediante semplici vincoli dichiarativi ed è stato quindi implementato tramite trigger.

Il controllo si basa su due costrutti nativi di PostgreSQL. Il primo è `TSRANGE(DATA_ORA_INIZIO, DATA_ORA_FINE)`, una funzione che trasforma una coppia di timestamp in un oggetto intervallo temporale: anziché considerare separatamente gli istanti di inizio e fine, il database tratta l'intera durata dell'attività come un'unica entità geometrica su cui è possibile eseguire operazioni di confronto spaziale. Il secondo è l'operatore `&&`, che applicato a due range restituisce vero se e solo se i due intervalli si sovrappongono, ovvero se esiste almeno un istante temporale appartenente a entrambi. Tale operatore consente di rilevare qualsiasi tipo di conflitto, sia che una attività ne contenga completamente un'altra, sia che si sovrapponga parzialmente solo all'inizio o alla fine.

La funzione `check_attivita` unifica tramite `UNION ALL` le quattro tabelle delle attività e, per ciascuna riga esistente, costruisce il relativo `TSRANGE`. Verifica quindi se almeno uno di questi intervalli soddisfa `&&` con quello della nuova attività: in caso affermativo, l'operazione viene rifiutata con un'eccezione esplicita. La `UNION ALL` anziché `UNION` è una scelta deliberata: non essendo necessaria l'eliminazione di duplicati (ogni attività ha il proprio intervallo distinto), evitando l'ordinamento implicito richiesto da `UNION` riduce il costo computazionale del controllo.

La clausola `AND (TG_OP='INSERT' OR t.data_ora_inizio <> OLD.data_ora_inizio)` evita che, in fase di `UPDATE`, la riga venga confrontata con sé stessa: poiché il trigger è `BEFORE`, la riga modificata è ancora presente nella tabella con i valori originali, e senza questa condizione il suo stesso intervallo precedente verrebbe rilevato come conflittuale con il nuovo. Per gli `INSERT` la condizione è invece sempre vera, poiché la riga non esiste ancora nella tabella al momento del controllo.

La funzione è stata associata, tramite trigger omonimi, a ciascuna delle quattro tabelle delle attività, così da garantire il controllo indipendentemente da quale categoria di attività venga inserita o modificata.

```

-- INSERT/UPDATE Attivita
CREATE OR REPLACE FUNCTION check_attivita()
RETURNS TRIGGER AS $$
DECLARE
    conflitto_rilevato BOOLEAN;
BEGIN

```



```

SELECT EXISTS (
  SELECT 1 FROM (
    SELECT data_ora_inizio, tsrange(data_ora_inizio, data_ora_fine)
      AS intervallo FROM attivita_routine
    UNION ALL
    SELECT data_ora_inizio, tsrange(data_ora_inizio, data_ora_fine)
      AS intervallo FROM attivita_faccende
    UNION ALL
    SELECT data_ora_inizio, tsrange(data_ora_inizio, data_ora_fine)
      AS intervallo FROM attivita_pasto
    UNION ALL
    SELECT data_ora_inizio, tsrange(data_ora_inizio, data_ora_fine)
      AS intervallo FROM attivita_extra
  ) t
 WHERE t.intervallo && tsrange(NEW.data_ora_inizio, NEW.data_ora_fine)
    AND (TG_OP = 'INSERT' OR t.data_ora_inizio <> OLD.data_ora_inizio)
) INTO conflitto_rilevato;

IF conflitto_rilevato THEN
  RAISE EXCEPTION 'Errore: L''intervallo temporale si sovrappone a un''attivit 
  gia pianificata!';
END IF;

RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

7.5.3 Vincolo di presenza dell'animatore responsabile nelle stanze degli animati

Il vincolo secondo cui ogni stanza che ospita animati deve contenere almeno un animatore responsabile non   esprimibile in modo dichiarativo, poich  coinvolge due tabelle distinte (Animati e Animatori) messe in relazione tramite i rispettivi alloggiamenti in partecipanti, ed   stato quindi implementato tramite trigger.

La funzione `check_stanza_animatore_responsabile_insert_update` viene invocata dopo ogni inserimento o modifica su animati. La funzione recupera la stanza del nuovo animato tramite partecipanti e verifica quanti animatori risultano assegnati alla medesima stanza: se il conteggio   diverso da uno, l'operazione viene rifiutata. Il trigger   definito come `CONSTRAINT TRIGGER DEFERRABLE INITIALLY DEFERRED` per le stesse ragioni gi  discusse per l'esclusivit  dei ruoli: l'assegnazione di un animato e del relativo animatore responsabile pu  avvenire mediante due operazioni distinte nella stessa transazione, e il controllo va effettuato solo a commit avvenuto, quando lo stato finale   definitivo.

```

-- INSERT/UPDATE
CREATE OR REPLACE FUNCTION check_stanza_animatore_responsabile_insert_update()
RETURNS TRIGGER AS $$
DECLARE
  cnt INTEGER;
BEGIN
  SELECT COUNT(*) INTO cnt
  FROM partecipanti p
  JOIN animatori a ON p.codice_fiscale = a.codice_fiscale
  WHERE p.numero_stanza IN (
    SELECT numero_stanza

```

```

    FROM partecipanti
    WHERE codice_fiscale = NEW.codice_fiscale
);

IF cnt <> 1 THEN
    RAISE EXCEPTION 'Ogni stanza deve avere esattamente un animatore responsabile';
END IF;

RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE CONSTRAINT TRIGGER trg_stanza_check_animatore_responsabile_presente
AFTER INSERT OR UPDATE ON animati
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW
EXECUTE FUNCTION check_stanza_animatore_responsabile_insert_update();

```

La funzione `check_stanza_animatore_responsabile_delete` gestisce il caso opposto: prima che un animatore venga eliminato, verifica se nella stessa stanza siano presenti animati. In caso affermativo, l'eliminazione viene bloccata, impedendo che una stanza con animati rimanga priva di responsabile. In questo caso il trigger è BEFORE standard e non differito: la verifica coinvolge una sola operazione atomica e il suo esito è immediatamente determinabile, senza necessità di attendere il completamento della transazione.

7.5.4 Ulteriori vincoli aziendali e di consistenza

Oltre ai trigger descritti in dettaglio, il progetto implementa i seguenti ulteriori vincoli procedurali:

- **Omogeneità di sesso nelle stanze degli animati:** prima dell'inserimento di un nuovo animato o della modifica della sua assegnazione a una stanza, il trigger verifica che tutti gli occupanti già presenti (animati e relativo animatore responsabile) abbiano lo stesso sesso del nuovo partecipante. Qualora venga rilevata una discordanza, l'operazione viene annullata mediante il lancio di un'eccezione, impedendo la formazione di stanze miste.
- **Capienza massima della stanza:** ad ogni inserimento o modifica dell'assegnazione di un partecipante a una stanza, il trigger conta il numero di occupanti già presenti e confronta tale valore con il numero di posti letto disponibili (`numero_posti_letto`). Qualora il limite risulti già raggiunto, l'operazione viene rifiutata.
- **Segregazione dei membri dell'equipe non animatori:** prima di assegnare un cuoco o un collaboratore a una stanza, verifica che in quella stanza non siano presenti animati. Il trigger impedisce che cuochi o collaboratori condividano una stanza con gli animati, garantendo il rispetto del vincolo previsto dalla traccia.
- **Monotonia crescente del punteggio squadra:** ad ogni aggiornamento del punteggio di una squadra, confronta il nuovo valore con quello precedentemente memorizzato (`OLD.punteggio`). Se il nuovo valore è inferiore o uguale, l'operazione viene rifiutata: il punteggio può solo crescere nel corso del campo.
- **Coerenza temporale inizio/fine attività:** BEFORE INSERT OR UPDATE su tutte le tabelle delle attività, verifica che `data_ora_fine` sia successiva a `data_ora_inizio`. Si tratta del controllo di consistenza temporale più elementare e può essere alternativamente espresso come vincolo CHECK a livello di tabella anziché come trigger.
- **Allergie del cuoco supervisore:** quando viene assegnato un cuoco come supervisore di un pasto, ricostruisce, mediante una sequenza di JOIN, la relazione tra pasto, pietanze e aller-

geni confrontando con le allergie registrate per quel cuoco. Se l'intersezione non è vuota, l'assegnazione viene bloccata.

- **Almeno un animato per squadra:** trigger AFTER DELETE su animati: se dopo la cancellazione la squadra di appartenenza risulta priva di animati, l'operazione viene rifiutata. Analogamente, un trigger AFTER UPDATE controlla che uno spostamento verso un'altra squadra non svuoti la squadra di provenienza.
- **Almeno una stanza per edificio:** trigger AFTER DELETE su stanze: se dopo la cancellazione l'edificio associato non ha più stanze, l'operazione viene bloccata.
- **Almeno una faccenda per squadra:** trigger AFTER DELETE su attivita_faccende (o sulla relazione che associa faccende e squadre): verifica che ogni squadra risulti ancora assegnata ad almeno una faccenda dopo la cancellazione.
- **Almeno una routine per animatore:** trigger AFTER DELETE o AFTER UPDATE sulla relazione che lega animatori e attività di routine: se l'animatore non risulta più responsabile di nessuna routine, l'operazione viene rifiutata.
- **Pietanza in almeno un pasto / Pasto con almeno una pietanza:** due trigger speculari sulla relazione pietanza_pasto. Il primo blocca la cancellazione di un'associazione se la pietanza coinvolta non compare in nessun altro pasto. Il secondo applica il medesimo controllo al pasto: se rimuovendo l'associazione il pasto rimane senza pietanze, l'operazione viene rifiutata.

7.6 Implementazione delle query ed esempi d'uso

A completamento della fase di progettazione e implementazione fisica, sono presentate le query SQL corrispondenti ad alcune delle attività necessarie allo svolgimento del campo-scuola. Le operazioni elencate di seguito rappresentano un insieme rappresentativo di attività significative, atte a coprire in maniera equilibrata le diverse aree dello schema.

L'implementazione pratica di queste interrogazioni dimostra la capacità della base di dati di rispondere in modo robusto ed efficiente alla gestione del campo-scuola.

7.6.1 Inserimento di un nuovo animato.

L'operazione è racchiusa in una transazione atomica che esegue tre INSERT distinte. La prima inserisce le informazioni anagrafiche comuni in partecipanti, comprensive dell'assegnazione del partecipante alla relativa stanza. La seconda specializza il partecipante in animati, collegandolo alla squadra di appartenenza e registrando i contatti del genitore. La terza popola la relazione allergico con gli allergeni associati. Si assume che stanza, squadra e allergeni siano già presenti nel database. La natura transazionale garantisce che, in caso di errore in uno qualsiasi dei tre passi, nessuna modifica venga persistita, evitando partecipanti privi di specializzazione o allergie registrate senza il relativo partecipante.

```
BEGIN;  
  
INSERT INTO partecipanti (codice_fiscale, nome, cognome, luogo_nascita,  
    foto_documento,  
    note, numero_stanza) VALUES  
( 'GRLMRC08E20H501T', 'Marco', 'Giraldi', 'Udine', 'foto_marco_giraldi.jpg',  
    'Allergico a glutine e lattosio', 101);  
  
INSERT INTO animati (codice_fiscale, email_genitore, telefono_genitore,  
    nome_squadra) VALUES  
( 'GRLMRC08E20H501T', 'parent1@email.com', '3301234567', 'Aquila Rosse');
```

```
INSERT INTO allergico (codice_fiscale_partecipante, nome_allergene) VALUES
('GRLMRC08E20H501T', 'glutine'),
('GRLMRC08E20H501T', 'lattosio');

COMMIT;
```

7.6.2 Inserimento di una nuova attività di routine.

Analogamente al caso precedente, la transazione esegue due INSERT. La prima inserisce l'attività in `attivita_routine`, specificando l'intervallo temporale, la tipologia, il nome e il codice fiscale dell'animatore supervisore. La seconda popola la relazione `routine_prevedono_materiali` con i materiali necessari e le relative quantità. Si assume che i materiali e l'animatore supervisore siano già presenti nel database. Il trigger di sequenzialità temporale viene eseguito automaticamente al momento del primo INSERT, garantendo che l'intervallo non si sovrapponga ad attività già pianificate.

```
BEGIN;

INSERT INTO attivita_routine (data_ora_inizio, data_ora_fine, descrizione, nome,
    codice_fiscale_animatore_supervisore, tipologia_routine) VALUES
('2026-05-12 15:00:00', '2026-05-12 17:00:00', 'Attività pomeriggio',
    'attività del pomeriggio', 'SFRNTN93E45B789L', 'attività del pomeriggio');

INSERT INTO routine_prevedono_materiali (data_ora_inizio_attivita, nome_materiale,
    quantita) VALUES
('2026-05-12 15:00:00', 'Pennarelli colorati', 3),
('2026-05-12 15:00:00', 'Quaderni', 10),
('2026-05-12 15:00:00', 'Materiale artistico', 2);

COMMIT;
```

7.6.3 Coppie di routine che condividono un materiale.

La query individua tutte le coppie di attività di routine che impiegano lo stesso materiale. Il self-join su `routine_prevedono_materiali` con la condizione `r1.data_ora_inizio_attivita < r2.data_ora_inizio_attivita` garantisce che ogni coppia compaia una sola volta nel risultato, evitando duplicati simmetrici. I JOIN successivi con `attivita_routine` recuperano il nome e l'orario di inizio di ciascuna delle due routine della coppia.

```
SELECT
    r1.nome_materiale AS materiale_condiviso,
    a1.data_ora_inizio AS inizio_routine_1,
    a1.nome AS nome_routine_1,
    a2.data_ora_inizio AS inizio_routine_2,
    a2.nome AS nome_routine_2
FROM routine_prevedono_materiali r1
JOIN routine_prevedono_materiali r2
    ON r1.nome_materiale = r2.nome_materiale
    AND r1.data_ora_inizio_attivita < r2.data_ora_inizio_attivita
JOIN attivita_routine a1
    ON r1.data_ora_inizio_attivita = a1.data_ora_inizio
JOIN attivita_routine a2
    ON r2.data_ora_inizio_attivita = a2.data_ora_inizio;
```

7.6.4 Cuochi che hanno supervisionato un pasto con un allergene presente in una sola pietanza.

La CTE `allergeni_singoli` individua preliminarmente gli allergeni che compaiono in una sola pietanza in tutto il database, tramite raggruppamento e filtro `HAVING COUNT = 1`. La query principale percorre poi la catena `pasto`, `pietanze`, `allergeni` e restringe i risultati ai soli cuochi supervisori di pasti che includono almeno una di queste pietanze rare. La clausola `DISTINCT` evita duplicazioni nel caso in cui lo stesso cuoco abbia supervisionato più pasti che soddisfano la condizione.

```
WITH allergeni_singoli AS (  
    SELECT nome_allergene  
    FROM pietanze_contengono_allergeni  
    GROUP BY nome_allergene  
    HAVING COUNT(nome_pietanza) = 1  
)  
SELECT DISTINCT  
    p.codice_fiscale,  
    p.nome,  
    p.cognome  
FROM partecipanti p  
JOIN cuochi c  
    ON p.codice_fiscale = c.codice_fiscale  
JOIN attivita_pasto ap  
    ON c.codice_fiscale = ap.codice_fiscale_cuoco_supervisore  
JOIN pietanze_per_pasto ppp  
    ON ap.data_ora_inizio = ppp.data_ora_inizio  
JOIN pietanze_contengono_allergeni pca  
    ON ppp.nome_pietanza = pca.nome_pietanza  
JOIN allergeni_singoli als  
    ON pca.nome_allergene = als.nome_allergene;
```

7.6.5 Animatore/i che ha supervisionato la routine con la maggiore quantità totale di materiale.

La CTE `routine_con_totale` calcola, per ogni attività di routine, la somma delle quantità di materiali previsti e assegna una posizione in classifica tramite `DENSE_RANK`. L'uso di `DENSE_RANK` anziché `ROW_NUMBER` consente di restituire tutti gli animatori a pari merito in caso di *ex aequo*. La query esterna filtra sulla prima posizione in classifica e recupera i dati anagrafici dell'animatore supervisore tramite `JOIN` con `partecipanti`.

```
WITH routine_con_totale AS (  
    SELECT  
        data_ora_inizio_attivita,  
        SUM(quantita) AS totale_quantita,  
        DENSE_RANK() OVER (ORDER BY SUM(quantita) DESC) AS classifica  
    FROM routine_prevedono_materiali  
    GROUP BY data_ora_inizio_attivita  
)  
SELECT  
    p.codice_fiscale,  
    p.nome,  
    p.cognome,  
    ar.nome AS nome_routine,  
    ar.data_ora_inizio,  
    r.totale_quantita  
FROM routine_con_totale r  
JOIN partecipanti p  
    ON p.codice_fiscale = ar.codice_fiscale_cuoco_supervisore  
WHERE r.classifica = 1;
```

```

JOIN attivita_routine ar
  ON r.data_ora_inizio_attivita = ar.data_ora_inizio
JOIN partecipanti p
  ON ar.codice_fiscale_animatore_supervisore = p.codice_fiscale
WHERE r.classifica = 1;

```

7.6.6 Animatori che hanno supervisionato solo attività senza materiali.

La query usa una doppia condizione esistenziale. Il primo EXISTS garantisce che l'animatore abbia effettivamente supervisionato almeno una routine, escludendo così gli animatori che non ne hanno gestita nessuna. Il NOT EXISTS esclude invece tutti gli animatori per cui esiste almeno una routine supervisionata con materiali associati nella relazione routine_prevedono_materiali. Il risultato comprende esclusivamente gli animatori attivi ma la cui totalità di routine non ha richiesto alcun materiale.

```

SELECT
  p.codice_fiscale,
  p.nome,
  p.cognome
FROM partecipanti p
JOIN animatori a
  ON p.codice_fiscale = a.codice_fiscale
WHERE EXISTS (
  -- L'animatore deve aver supervisionato almeno un'attività di routine
  SELECT 1
  FROM attivita_routine ar
  WHERE ar.codice_fiscale_animatore_supervisore = a.codice_fiscale
)
AND NOT EXISTS (
  -- NON deve esistere alcuna attività da lui supervisionata che richieda materiali
  SELECT 1
  FROM attivita_routine ar
  JOIN routine_prevedono_materiali rpm
    ON ar.data_ora_inizio = rpm.data_ora_inizio_attivita
  WHERE ar.codice_fiscale_animatore_supervisore = a.codice_fiscale
);

```

7.6.7 Aggiornamento dell'orario di una routine.

La query modifica i campi data_ora_inizio e data_ora_fine di una specifica attività di routine, identificata dal suo orario di inizio corrente. Il trigger di sequenzialità temporale viene invocato automaticamente sull'UPDATE, verificando che il nuovo intervallo non si sovrapponga ad altre attività già pianificate; la condizione t.data_ora_inizio <> OLD.data_ora_inizio al suo interno esclude la riga stessa dal confronto, evitando falsi conflitti con l'intervallo originario.

```

UPDATE attivita_routine SET
  data_ora_inizio = '2026-07-20 16:00:00',
  data_ora_fine = '2026-07-20 17:30:00'
WHERE
  data_ora_inizio = '2026-07-20 09:30:00';

```

7.6.8 Cancellazione di una routine.

La cancellazione è espressa da un singolo DELETE su `attivit _routine`, identificando la riga tramite la chiave primaria `data_ora_inizio`. Grazie alla clausola `ON DELETE CASCADE` definita sulle chiavi esterne, tutte le righe dipendenti nella relazione `routine_prevedono_materiali` sono eliminate automaticamente, senza necessit  di istruzioni aggiuntive. Il trigger di cardinalit  sull'animatore supervisore viene invocato automaticamente, bloccando la cancellazione qualora questa lasciasse l'animatore privo di qualsiasi routine supervisionata.

```
DELETE FROM attivita_routine WHERE data_ora_inizio = '2026-07-20 16:00:00';
```

8 Conclusioni e sviluppi futuri

Il progetto ha consentito di sviluppare con successo una base di dati relazionale completa per la gestione di un campo-scuola, affrontando tutte le fasi della progettazione:

- **Analisi e progettazione concettuale:** l'analisi della traccia ha condotto alla formulazione delle assunzioni iniziali e alla definizione di tre macroaree funzionali, a partire dalle quali   stato costruito il diagramma ER.
- **Progettazione logica:** lo schema concettuale   stato ristrutturato eliminando tutti i costrutti non esprimibili nello schema logico (generalizzazioni, relazioni molti-a-molti, attributi composti) e valutando i costi computazionali derivanti dall'inserimento di una ridondanza.
- **Progettazione fisica:** tramite il RDBMS PostgreSQL, sono state create le tabelle, in conformit  con quanto dichiarato nello schema logico-relazionale, e gli indici strategici per l'ottimizzazione delle prestazioni.
- **Implementazione vincoli e interrogazioni:** a seguito dell'individuazione dei principali trigger da implementare per garantire il corretto funzionamento della base di dati, si sono fornite le implementazioni dei vincoli pi  significativi. Per valutare il corretto funzionamento della base sono state infine fornite delle interrogazioni.

Il risultato finale   una base di dati coerente, robusta e in grado di supportare efficacemente la gestione delle attivit  previste dalla traccia progettuale.

Nel corso dello sviluppo, le fasi di progettazione ed implementazione del database sono state documentate e versionate all'interno di un repository GitHub, reperibile a questo [indirizzo](#).

Nonostante il risultato raggiunto sia allineato con l'obiettivo della traccia, il sistema presenta ampi margini di miglioramento in ottica di una futura applicazione in scenari reali. Alcune fra le principali evoluzioni possibili sono elencate di seguito:

- Estensione del tracciamento delle informazioni per gestire pi  campi scuola storicizzati nel tempo;
- Adozione di tabelle di lookup al posto degli enum per superarne i limiti e rendere l'intera base di dati pi  portabile;
- Gestione avanzata dei materiali e delle pietanze, includendo anche gli ingredienti base e i costi d'acquisto;
- Implementazione dei meccanismi di sicurezza per la protezione dei dati sensibili (foto documenti, informazioni sanitarie) e per garantire conformit  con la normativa GDPR.